

SPRAWOZDANIE

Zajęcia: Grafika i gry komputerowe I
Prowadzący: mgr. inż. Jakub Krawczyk

Laboratorium 7

Data 29.04.26r

Temat:

„Podstawy WebGL/GLSL”

Sprawozdanie :

[https://github.com/Kamalo00/Grafika-](https://github.com/Kamalo00/Grafika-komputerowa-pliki.git)
[komputerowa-pliki.git](https://github.com/Kamalo00/Grafika-komputerowa-pliki.git)

Kamil Greń
Informatyka I stopień,
stacjonarnie,
4 semestr,
Gr.1b

ZADANIE 1

1. Polecenie :

Zmodyfikować plik lab11.html , aby wyświetlał ruchome punkty (kwadraty) odbijające się od krawędzi płótna.

Przypisanie losowego koloru każdemu punktowi z możliwością zmiany koloru oraz dodanie innych stylów wyświetlania : wielokąty , linie itd.

2. Wprowadzane dane:

Menu do zmiany parametrów :

- Spacja – pauza / wznowienie animacji
- C – przełączanie kolorów figur
- W – zmiana liczby wierzchołków wielokąta
- K – tryb linii
- 1/2/3/4 – wybór stylu dla punktów

3. Wykorzystane komendy:

```
<script type="x-shader/x-fragment" id="fshader-source">
    #ifdef GL_FRAGMENT_PRECISION_HIGH
        precision highp float;
    #else
        precision mediump float;
    #endif

    varying vec3 v_color;
    uniform int u_style;
    uniform int u_sides;

    const float PI = 3.14159265;

    float insidePolygon(vec2 p, float n, float radius) {
        float angle = atan(p.y, p.x);
        float sector = PI * 2.0 / n;
        float a = mod(angle, sector) - sector * 0.5;
        float dist = cos(a) * length(p);
        return dist < radius ? 1.0 : 0.0;
    }

    void main() {
        vec2 coord = gl_PointCoord - vec2(0.5);
        float dist = length(coord);

        if (u_style == 1) {
            gl_FragColor = vec4(v_color, 1.0);
        } else if (u_style == 2) {
            if (dist > 0.5) discard;
            gl_FragColor = vec4(v_color, 1.0);
        } else if (u_style == 3) {
            float n = float(u_sides);
            if (insidePolygon(coord, n, 0.45) < 0.5) discard;
            gl_FragColor = vec4(v_color, 1.0);
        } else if (u_style == 4) {
            float n = float(u_sides);
            float angle = atan(coord.y, coord.x);
            float sector = PI * 2.0 / n;
            float a = mod(angle, sector) - sector * 0.5;
            float edge = cos(a) * 0.45;
            float t = dist / max(edge, 0.001);
            if (t > 1.0) discard;
            float alpha = 1.0 - t;
            gl_FragColor = vec4(v_color, alpha);
        } else {
            gl_FragColor = vec4(v_color, 1.0);
        }
    }
</script>
```

```

108 <script>
109     "use strict";
110
111     var canvas, gl;
112
113     var u_width_loc, u_height_loc, u_pointSize_loc;
114     var u_style_loc, u_sides_loc;
115     var a_coords_loc, a_color_loc;
116
117     var a_coords_buffer, a_color_buffer, line_buffer;
118
119     var POINT_COUNT = 200;
120     var POINT_SIZE = 32;
121
122     var positions = new Float32Array(2 * POINT_COUNT);
123     var velocities = new Float32Array(2 * POINT_COUNT);
124     var colors = new Float32Array(3 * POINT_COUNT);
125
126     var isRunning = true;
127     var isColorRandom = false;
128     var isDrawLines = false;
129     var pointStyle = 1;
130     var nSides = 6;
131
132     function render() {
133         gl.clear(gl.COLOR_BUFFER_BIT);
134
135         if (isDrawLines) {
136             var linePos = buildLinePositions();
137             gl.bindBuffer(gl.ARRAY_BUFFER, line_buffer);
138             gl.bufferData(gl.ARRAY_BUFFER, linePos, gl.STREAM_DRAW);
139             gl.vertexAttribPointer(a_coords_loc, 2, gl.FLOAT, false, 0, 0);
140
141             if (isColorRandom) {
142                 gl.bindBuffer(gl.ARRAY_BUFFER, a_color_buffer);
143                 gl.enableVertexAttribArray(a_color_loc);
144                 gl.vertexAttribPointer(a_color_loc, 3, gl.FLOAT, false, 0, 0);
145             } else {
146                 gl.disableVertexAttribArray(a_color_loc);
147                 gl.vertexAttrib3f(a_color_loc, 1.0, 0.0, 0.0);
148             }
149
150             gl.uniform1i(u_style_loc, 1);
151             gl.drawArrays(gl.LINES, 0, linePos.length / 2);
152
153         } else {
154             gl.bindBuffer(gl.ARRAY_BUFFER, a_coords_buffer);
155             gl.bufferData(gl.ARRAY_BUFFER, positions, gl.STREAM_DRAW);
156             gl.vertexAttribPointer(a_coords_loc, 2, gl.FLOAT, false, 0, 0);
157
158             if (isColorRandom) {
159                 gl.bindBuffer(gl.ARRAY_BUFFER, a_color_buffer);
160                 gl.enableVertexAttribArray(a_color_loc);
161                 gl.vertexAttribPointer(a_color_loc, 3, gl.FLOAT, false, 0, 0);
162             } else {
163                 gl.disableVertexAttribArray(a_color_loc);
164                 gl.vertexAttrib3f(a_color_loc, 1.0, 0.0, 0.0);
165             }
166
167             gl.uniform1i(u_style_loc, pointStyle);
168             gl.uniform1i(u_sides_loc, nSides);
169             gl.drawArrays(gl.POINTS, 0, POINT_COUNT);
170         }
171     }
172

```

```

function buildLinePositions() {
    var arr = new Float32Array(4 * (POINT_COUNT - 1));
    for (var i = 0; i < POINT_COUNT - 1; i++) {
        arr[4*i] = positions[2*i];
        arr[4*i+1] = positions[2*i+1];
        arr[4*i+2] = positions[2*(i+1)];
        arr[4*i+3] = positions[2*(i+1)+1];
    }
    return arr;
}

function updateFrame() {
    for (var i = 0; i < POINT_COUNT; i++) {
        positions[2*i] += velocities[2*i];
        positions[2*i+1] += velocities[2*i+1];

        if ((positions[2*i] < 0 && velocities[2*i] < 0) ||
            (positions[2*i] > canvas.width && velocities[2*i] > 0)) {
            velocities[2*i] = -velocities[2*i];
        }
        if ((positions[2*i+1] < 0 && velocities[2*i+1] < 0) ||
            (positions[2*i+1] > canvas.height && velocities[2*i+1] > 0)) {
            velocities[2*i+1] = -velocities[2*i+1];
        }
    }
}

```

```

function doKey(evt) {
    var code = evt.key;

    if (code === " ") {
        isRunning = !isRunning;
        return;
    }
    if (code === "c" || code === "C") {
        isColorRandom = !isColorRandom;
        if (isColorRandom) randomizeColors();
        return;
    }
    if (code === "w" || code === "W") {
        var num = prompt("Podaj liczbę wierzchołków wielokąta:", nSides);
        if (num !== null) {
            nSides = Math.max(3, parseInt(num) || 3);
            gl.uniform1i(u_sides_loc, nSides);
        }
        return;
    }
    if (code === "k" || code === "K") {
        isDrawLines = !isDrawLines;
        return;
    }
    if (code === "1") { pointStyle = 1; return; }
    if (code === "2") { pointStyle = 2; return; }
    if (code === "3") { pointStyle = 3; return; }
    if (code === "4") { pointStyle = 4; return; }
}

function doMouseDown(evt) {
    if (evt.shiftKey) {
        createData();
        return;
    }
    var r = evt.target.getBoundingClientRect();
    var mx = evt.clientX - r.left;
    var my = evt.clientY - r.top;
    for (var i = 0; i < POINT_COUNT; i++) {
        var dx = mx - positions[2*i];
        var dy = my - positions[2*i+1];
        var dist = Math.sqrt(dx*dx + dy*dy);
        if (dist < 0.001) continue;
        var spd = Math.sqrt(velocities[2*i]*velocities[2*i] + velocities[2*i+1]*velocities[2*i+1]);
        velocities[2*i] = spd * dx / dist;
        velocities[2*i+1] = spd * dy / dist;
    }
}

function randomizeColors() {
    for (var i = 0; i < colors.length; i++) colors[i] = Math.random();
    gl.bindBuffer(gl.ARRAY_BUFFER, a_color_buffer);
    gl.bufferData(gl.ARRAY_BUFFER, colors, gl.STATIC_DRAW);
}

```

```

248
249
250     function randomizeColors() {
251         for (var i = 0; i < colors.length; i++) colors[i] = Math.random();
252         gl.bindBuffer(gl.ARRAY_BUFFER, a_color_buffer);
253         gl.bufferData(gl.ARRAY_BUFFER, colors, gl.STATIC_DRAW);
254     }
255
256     function createData() {
257         for (var i = 0; i < 3 * POINT_COUNT; i++) colors[i] = Math.random();
258
259         for (var i = 0; i < POINT_COUNT; i++) {
260             positions[2*i] = canvas.width / 2;
261             positions[2*i+1] = canvas.height / 2;
262             var speed = 2 + 4 * Math.random();
263             var angle = 2 * Math.PI * Math.random();
264             velocities[2*i] = speed * Math.cos(angle);
265             velocities[2*i+1] = speed * Math.sin(angle);
266         }
267     }
268
269     function initGL() {
270         gl.viewport(0, 0, canvas.width, canvas.height);
271         gl.clearColor(0.2, 0.2, 0.2, 1.0);
272
273         gl.enable(gl.BLEND);
274         gl.blendFunc(gl.SRC_ALPHA, gl.ONE_MINUS_SRC_ALPHA);

```

```

function compileShader(type, sourceId) {
    var sh = gl.createShader(type);
    gl.shaderSource(sh, document.getElementById(sourceId).text);
    gl.compileShader(sh);
    if (!gl.getShaderParameter(sh, gl.COMPILE_STATUS)) {
        console.error("Shader error:", gl.getShaderInfoLog(sh));
        return null;
    }
    return sh;
}

var vsh = compileShader(gl.VERTEX_SHADER, "vshader-source");
var fsh = compileShader(gl.FRAGMENT_SHADER, "fshader-source");

var prog = gl.createProgram();
gl.attachShader(prog, vsh);
gl.attachShader(prog, fsh);
gl.linkProgram(prog);
if (!gl.getProgramParameter(prog, gl.LINK_STATUS)) {
    console.error("Link error:", gl.getProgramInfoLog(prog));
    return;
}
gl.useProgram(prog);

a_coords_loc = gl.getAttribLocation(prog, "a_coords");
a_color_loc = gl.getAttribLocation(prog, "a_color");
u_width_loc = gl.getUniformLocation(prog, "u_width");
u_height_loc = gl.getUniformLocation(prog, "u_height");
u_pointSize_loc = gl.getUniformLocation(prog, "u_pointSize");
u_style_loc = gl.getUniformLocation(prog, "u_style");
u_sides_loc = gl.getUniformLocation(prog, "u_sides");

a_coords_buffer = gl.createBuffer();
gl.bindBuffer(gl.ARRAY_BUFFER, a_coords_buffer);
gl.enableVertexAttribArray(a_coords_loc);

a_color_buffer = gl.createBuffer();
gl.bindBuffer(gl.ARRAY_BUFFER, a_color_buffer);
gl.bufferData(gl.ARRAY_BUFFER, colors, gl.STATIC_DRAW);
gl.disableVertexAttribArray(a_color_loc);
gl.vertexAttrib3f(a_color_loc, 1.0, 0.0, 0.0);

line_buffer = gl.createBuffer();

gl.uniform1f(u_width_loc, canvas.width);
gl.uniform1f(u_height_loc, canvas.height);
gl.uniform1f(u_pointSize_loc, POINT_SIZE);
gl.uniform1i(u_style_loc, pointStyle);
gl.uniform1i(u_sides_loc, nSides);
}

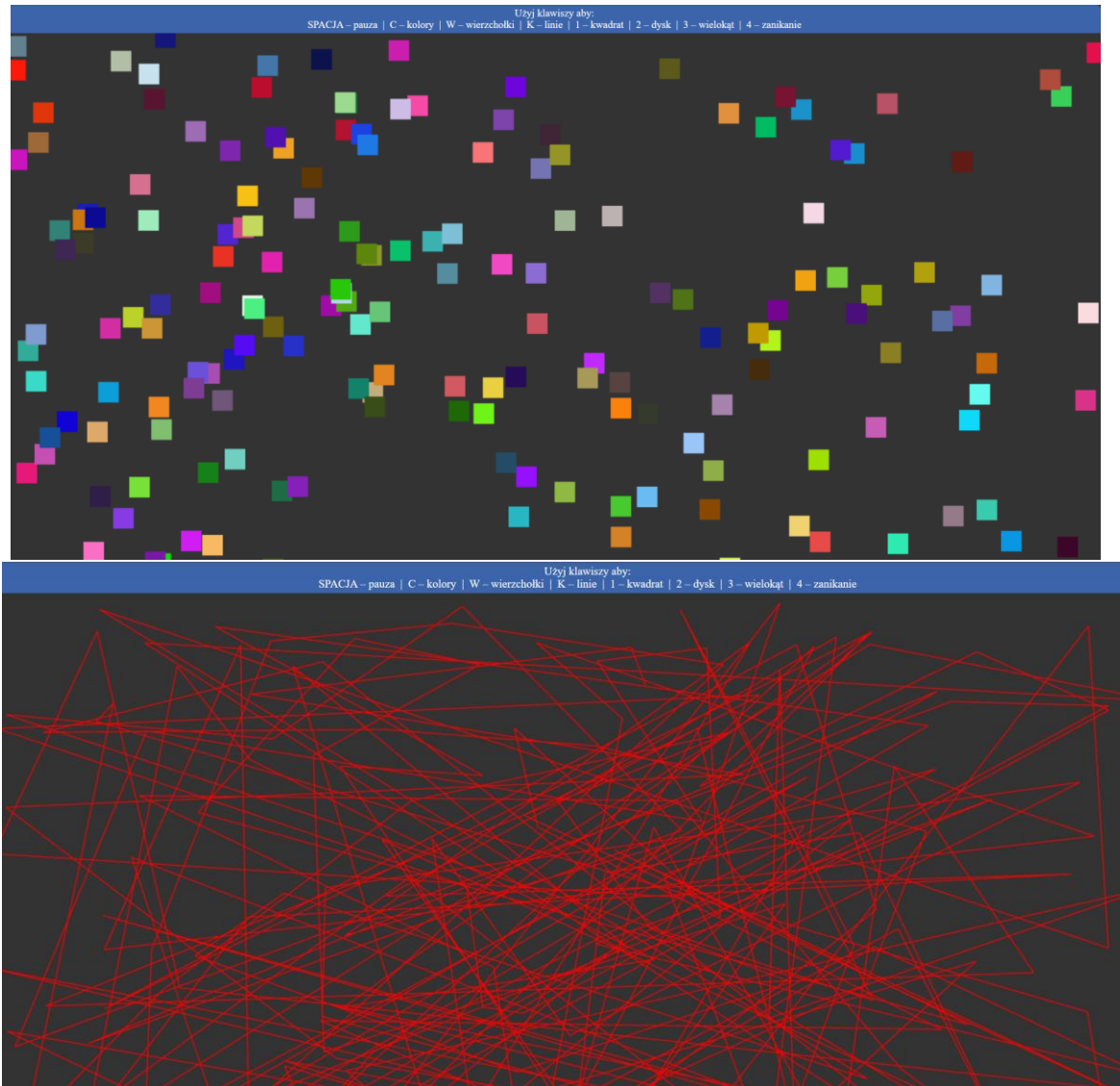
```

```

325
326
327     function init() {
328         try {
329             canvas = document.createElement("canvas");
330             document.body.appendChild(canvas);
331             canvas.width = window.innerWidth;
332             canvas.height = window.innerHeight;
333             gl = canvas.getContext("webgl");
334             if (!gl) throw "Przeglądarka nie obsługuje WebGL";
335         } catch(e) {
336             document.body.innerHTML = "<b>Błąd: " + e + "</b>";
337             return;
338         }
339
340         createData();
341         initGL();
342
343         canvas.addEventListener("mousedown", doMouseDown);
344         window.addEventListener("keydown", doKey);
345         window.addEventListener("resize", function() {
346             canvas.width = window.innerWidth;
347             canvas.height = window.innerHeight;
348             gl.viewport(0, 0, canvas.width, canvas.height);
349             gl.uniform1f(u_width_loc, canvas.width);
350             gl.uniform1f(u_height_loc, canvas.height);
351         });
352
353         var lastTime = 0;
354         function doFrame(time) {
355             if (isRunning && lastTime !== 0) {
356                 updateFrame();
357                 render();
358             } else if (!isRunning) {
359                 render();
360             } else {
361                 render();
362             }
363             lastTime = time;
364             requestAnimationFrame(doFrame);
365         }
366         requestAnimationFrame(doFrame);
367
368         window.addEventListener("load", init);
369     }
370 </script>
371 </head>
372 <body></body>
373 </html>

```

4. Wynik działania:



5. Wnioski:

Atrybut `attribute` pozwala na przypisanie innej wartości dla każdego wierzchołka, dzięki czemu każdemu punktowi można nadać inny kolor. Varying to mechanizm przekazujący dane z vertex shadera aż do fragmentu shadera, jego wartości są automatycznie interpolowane przez GPU pomiędzy wierzchołkami.

`gl_PointCoord` – wbudowana zmienna dostępna tylko w fragmencie shadera, umożliwia matematyczne „wycinanie” dowolnych kształtów.